

Lab 05 — Multi-stage en images van productiekwaliteit

Theorie — hoe je van een image van 500 MB die je compiler bevat, naar een image van 200 MB gaat die alleen bevat wat draait; en wat Buildah doet in plaats van BuildKit.

Doelstellingen

- Begrijpen waarom een image die dient om te **bouwen** niet die mag zijn die **uitvoert**.
- Een **multi-stage** build schrijven voor Spring Boot en voor Angular.
- Met kennis van zaken een basisimage kiezen (Debian/Ubuntu, Alpine, distroless).
- Weten wat BuildKit (Docker) en Buildah (Podman) bieden: cache, geheimen, stages.
- Imagegrootte, aanvalsoppervlak en uitroltijd met elkaar verbinden.

1. Het probleem: de buildtooling blijft in de image

Een eerste naïeve Dockerfile voor een Java-API:

```
FROM docker.io/library/maven:3.9-eclipse-temurin-21
WORKDIR /app
COPY . .
RUN mvn package -DskipTests
ENTRYPOINT ["java", "-jar", "/app/target/api.jar"]
```

Het werkt. Het levert een image van **800 MB tot 1 GB** met: een volledige JDK (compiler, debugtools), Maven, de lokale repository `~/.m2` met honderden JAR's, je **broncode**, de tests, en de uiteindelijke JAR. In productie dienen alleen de JRE en de JAR: ongeveer 200 MB.

De gevolgen zijn niet alleen esthetisch:

- **Beveiliging.** De broncode gaat naar iedereen die de image heeft. Elke meegeleverde tool (compiler, `curl`, `git`, shell) is een bijkomend actiemiddel voor een aanvaller, en een regel meer in het kwetsbaarheidsrapport.
- **Kost.** 800 MB overgedragen bij elke uitrol, naar elke node, opgeslagen in elke registry, met retentie van de vorige versies.
- **Tijd.** Een container starten omvat het downloaden van de image als ze ontbreekt. Bij een incident om 3 uur 's nachts is het verschil merkbaar.

→ BEVEILIGING

Het **aanvalsoppervlak** van een image is alles wat een aanvaller kan *gebruiken* zodra hij code kan uitvoeren: een shell om te verkennen, `curl` om te exfiltreren, een compiler, een pakketbeheerder. Elk afwezig binary is een stap meer voor hem. Daarom tellen scanners (Trivy, Gype) pakketten, en gaat "minimaal" niet alleen over megabytes.

2. Multi-stage

Een Dockerfile kan **meerdere FROM's** bevatten. Elk opent een *stage* — een onafhankelijke bouwomgeving. Alleen de **laatste** levert de uiteindelijke image; de andere worden weggegooid.

En `COPY --from=<stage>` laat toe bestanden op te halen uit een vorige stage.

```
# ----- stage 1: build -----
FROM docker.io/library/maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn -q dependency:go-offline
COPY src ./src
RUN mvn -q package -DskipTests

# ----- stage 2: runtime -----
FROM docker.io/library/eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=build /app/target/api.jar app.jar
USER 1000:1000
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

De uiteindelijke image bevat **een JRE en een JAR**. Geen Maven, JDK, bronnen, tests of `.m2`. Niets van wat in de stage `build` gebeurde, laat er een spoor in na — zelfs niet in verborgen lagen, want die lagen maken gewoon geen deel uit van de image.

ONTHOUDEN

Multi-stage is ook de enige echt betrouwbare bescherming tegen buildgeheimen: wat in een weggegooid stage gekopieerd wordt, bestaat niet in de uiteindelijke image. Let wel: `COPY --from=build /app /app` zou alles kopiëren, geheimen inbegrepen. Je kopieert alleen het artefact.

Hetzelfde schema voor Angular:

```
FROM docker.io/library/node:22-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build # levert dist/

FROM docker.io/library/nginx:alpine
COPY --from=build /app/dist/mijn-app/browser /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
```

→ ANGULAR

`ng build` compileert de TypeScript-componenten, bundelt alles in enkele `.js`-, `.css`-bestanden en een `index.html`, minimaliseert ze en geeft ze een vingerafdruk in hun naam (`main-a1b2c3.js`) voor de browsercache. Het resultaat is **statisch**: eender welke bestandserver serveert het. `ng serve` is een ontwikkelservers die bij elke wijziging hercompileert — kostbaar op je werkpost, zinloos in productie.

Cruciaal punt: **Node overleeft** de build **niet**. Een Angular-frontend in productie is alleen HTML, CSS en JavaScript, zonder JavaScript-engine aan serverzijde. Je containeriseert **nooit** `ng serve`.

3. Je basisimage kiezen

Basis	Grootte	Voordelen	Nadelen
<code>debian / ubuntu</code>	75-120 MB	Alles werkt, volledige tooling, <code>glibc</code>	Zwaar, veel pakketten dus veel CVE's
<code>*-slim</code>	30-80 MB	Goed compromis, blijft Debian	Minder tools geïnstalleerd
<code>alpine</code>	5-10 MB	Heel licht, efficiënte <code>apk</code>	Gebruikt musl en niet <code>glibc</code>
<code>distroless</code>	20-50 MB	Geen shell, geen pakketbeheerder	Moeilijk te debuggen, geen <code>exec sh</code>

→ LINUX

De **C-bibliotheek** (`libc`) is de laag tussen de programma's en de kernel: `printf`, `malloc`, DNS-resolutie, locales. Bijna elk Linux-binary hangt ervan af. `glibc` (GNU) is de historische, rijke, compatibele implementatie; `musl` een minimalistische herschrijving, door Alpine gekozen om zijn omvang. Een binary gecompileerd voor de ene laadt niet met de andere: `ldd --version` in de container zegt je welke je hebt.

De Alpine-valkuil verdient uitleg. De meeste programma's kunnen om met `musl`, maar niet allemaal: native binaries gecompileerd voor `glibc` weigeren te starten, sommige native Java-bibliotheken (compressie, cryptografie, PDF-generatie) falen met `UnsatisfiedLinkError`, en er duiken verschillen op in DNS-resolutie of locales.

In de praktijk, voor Spring Boot: `eclipse-temurin:21-jre-alpine` volstaat in de overgrote meerderheid van de gevallen en halveert de grootte; bij een native afhankelijkheid keer je terug naar `eclipse-temurin:21-jre` (Ubuntu). De keuze wordt **getest**, niet beslist.

Distroless images (Google) bevatten alleen de runtime en je applicatie: geen shell, geen `ls`, geen pakketbeheerder. Het aanvalsoppervlak is minimaal, maar `podman exec -it container sh` werkt niet meer — voorzie je observeerbaarheid anders.

4. Wat echt weegt

Vier hefbomen, in dalende volgorde van doeltreffendheid: (1) **multi-stage** — schrapt de buildtooling, de grote hefboom: 800 MB → 200 MB; (2) **de basisimage** — `-jre` in plaats van `-jdk`, `alpine` in plaats van `ubuntu`; (3) **.dockerignore** — geen `.git`, `node_modules`, `target` in de image; (4) **installatie en opruiming groeperen** in dezelfde `RUN`.

Wat daarentegen **geen enkel** effect heeft: bestanden verwijderen in een latere laag — ze blijven in de image (lab 02). En het aantal lagen op zich verandert bijna niets aan de grootte.

```
podman images --format 'table {{.Repository}}\t{{.Tag}}\t{{.Size}}'\npodman history mijn-api:1.0 --format 'table {{.Size}}\t{{.CreatedBy}}' | head
```

5. BuildKit en Buildah

Docker bouwt met **BuildKit**; Podman bouwt met **Buildah**. Beide lezen dezelfde Dockerfile en bieden dezelfde nuttige functies:

- **Ongebruikte stages worden niet gebouwd.** Een stage waaruit niets naar de uiteindelijke image gekopieerd wordt, wordt overgeslagen — Buildah toont `[2/3]`, `[3/3]` en slaat `[1/3]` over.

- **--target** bouwt tot een gegeven stage: `podman build --target build -t api-build .` geeft je de image van de compilatiestage, om ze te inspecteren.
- **Persistente caches.** `RUN --mount=type=cache,target=/root/.m2 mvn package` bewaart de Maven-repository **tussen builds**, zonder ze in de image op te nemen. Op een CI-agent is de winst spectaculair.
- **Geheimen.** `RUN --mount=type=secret,id=npnrc ...` maakt een bestand beschikbaar tijdens één enkele instructie, zonder het ooit in een laag te schrijven (lab 08).

```
FROM docker.io/library/maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN --mount=type=cache,target=/root/.m2 mvn -q package -DskipTests
```

PODMAN

Een zichtbaar verschil: BuildKit bouwt onafhankelijke stages **parallel**, Buildah na elkaar. Een ander: de regel `# syntax=docker/dockerfile:1`, die bij Docker de uitgebreide syntax activeert, wordt door Buildah gewoon **genegeerd** — de `--mount` s werken zonder. En de cache van `type=cache` leeft in je gebruikersopslag (`~/.local/share/containers/storage`), niet in een daemon: twee gebruikers van dezelfde CI-server delen hem niet.

6. Spring Boot: de lagen van de JAR

Een Spring Boot-JAR weegt 50 MB: 45 MB afhankelijkheden die bijna nooit veranderen en 5 MB code die bij elke commit verandert. In één blok gekopieerd vormt hij één laag van 50 MB die bij elke uitrol volledig opnieuw wordt overgedragen. Spring Boot kan zichzelf opdelen:

```
FROM docker.io/library/eclipse-temurin:21-jre-alpine AS extract
WORKDIR /app
COPY target/api.jar api.jar
RUN java -Djarmode=tools -jar api.jar extract --layers --destination extracted

FROM docker.io/library/eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=extract /app/extracted/dependencies/ ./
COPY --from=extract /app/extracted/spring-boot-loader/ ./
COPY --from=extract /app/extracted/snapshot-dependencies/ ./
COPY --from=extract /app/extracted/application/ ./
ENTRYPOINT ["java", "-jar", "app.jar"]
```

De afhankelijkheden vormen een stabiele laag, de code een kleine vluchtige laag: de uitrol draagt nog maar enkele MB over. Onthoud het **principe**: dat van lab 04, toegepast op de inhoud van een JAR.

7. In het bedrijf

- **Eén Dockerfile per dienst**, multi-stage, geversioneerd met de code. De CI heeft Maven noch Node nodig: `podman build` (of `docker build`) volstaat, zodat de build van de CI en die van de werkpost identiek zijn.

- **De tests** draaien vaak in een aparte stage (`RUN mvn test`), zodat een rode test de build van de image doet mislukken.
 - **De kwetsbaarheidsscan** (Trivy, Gype) gebeurt op de uiteindelijke image. Een minimale image levert een kort rapport, dat dus echt behandeld wordt — een image van 1 GB levert 300 CVE's die niemand leest.
 - **De uiteindelijke image draait als niet-root**, op een poort > 1024, zo mogelijk zonder shell.
-

Onthouden

- Wat bouwt, mag niet uitvoeren: dat is het hele punt van multi-stage.
- Meerdere `FROM`s = meerdere stages; alleen de laatste wordt de image, `COPY --from` haalt er het artefact in.
- Node heeft niets te zoeken in de uiteindelijke image van een Angular-frontend: statische inhoud wordt geserveerd door nginx.
- `-jre` eerder dan `-jdk`, `alpine` als de native afhankelijkheden het toelaten, distroless als je aanvaardt de shell te verliezen.
- Alpine gebruikt `musl` en niet `glibc`: te valideren met een test, nooit uit principe.
- BuildKit en Buildah bieden `--target`, persistente caches en buildgeheimen; Buildah negeert `#syntax=` en paralleliseert niet. Een bestand verwijderen in een latere laag maakt de image niet kleiner.

Woordenschat

stage: bouwstap geopend door een `FROM`. — `COPY --from`: bestanden ophalen uit een andere stage of image. — `--target`: de build stoppen bij een stage. — **distroless**: image zonder shell of pakketbeheerder. — **musl / glibc**: twee implementaties van de C-bibliotheek. — **BuildKit / Buildah**: build-engines van Docker en Podman. — **cache mount**: persistente cache tussen builds, buiten de image. — **aanvalsoppervlak**: de exploiteerbare componenten in de image.